

Тестирование roles

 [Bookmark](#)

Домашнее задание: Тестирование roles

Подготовка к выполнению

1. Установите molecule и его драйвера: `pip3 install "molecule molecule_docker molecule_podman"`.
2. Выполните `docker pull aragast/netology:latest` — это образ с podman, tox и несколькими пайтонами (3.7 и 3.9) внутри.

Основная часть

Ваша цель — настроить тестирование ваших ролей.

Задача — сделать сценарии тестирования для vector.

Ожидаемый результат — все сценарии успешно проходят тестирование ролей.

Molecule

1. Запустите `molecule test -s ubuntu_xenial` (или с любым другим сценарием, не имеет значения) внутри корневой директории clickhouse-role, посмотрите на вывод команды. Данная команда может отработать с ошибками или не отработать вовсе, это нормально. Наша цель - посмотреть как другие в реальном мире используют молекулу И из чего может состоять сценарий тестирования.
2. Перейдите в каталог с ролью vector-role и создайте сценарий тестирования по умолчанию при помощи `molecule init scenario --driver-name docker`.
3. Добавьте несколько разных дистрибутивов (oraclelinux:8, ubuntu:latest) для инстансов и протестируйте роль, исправьте найденные ошибки, если они есть.
4. Добавьте несколько assert в verify.yml-файл для проверки работоспособности vector-role (проверка, что конфиг валидный, проверка успешности запуска и др.).
5. Запустите тестирование роли повторно и проверьте, что оно прошло успешно.
6. Добавьте новый тег на коммит с рабочим сценарием в соответствии с семантическим версионированием.

Tox

1. Добавьте в директорию с vector-role файлы из [директории](#).
2. Запустите `docker run --privileged=True -v <path_to_repo>:/opt/vector-role -w /opt/vector-role -it aragast/netology:latest /bin/bash`, где path_to_repo — путь до корня репозитория с vector-role на вашей файловой системе.
3. Внутри контейнера выполните команду `tox`, посмотрите на вывод.
4. Создайте облегчённый сценарий для `molecule` с драйвером `molecule_podman`. Проверьте его на исполнимость.
5. Пропишите правильную команду в `tox.ini`, чтобы запускался облегчённый сценарий.
6. Запустите команду `tox`. Убедитесь, что всё отработало успешно.
7. Добавьте новый тег на коммит с рабочим сценарием в соответствии с семантическим версионированием.

После выполнения у вас должно получиться два сценария molecule и один tox.ini файл в репозитории. Не забудьте указать в ответе теги решений `Tox` и `Molecule` заданий. В качестве решения пришлите ссылку на ваш репозиторий и скриншоты этапов выполнения задания.

Необязательная часть

1. Прodelайте схожие манипуляции для создания роли `LightHouse`.
2. Создайте сценарий внутри любой из своих ролей, который умеет поднимать весь стек при помощи всех ролей.
3. Убедитесь в работоспособности своего стека. Создайте отдельный `verify.yml`, который будет проверять работоспособность интеграции всех инструментов между ними.
4. Выложите свои `roles` в репозитории.

В качестве решения пришлите ссылки и скриншоты этапов выполнения задания.

Разбор решения задания.

Настройка рабочего окружения:

- Основная рабочая директория — `~/ansible/vector-role`, как продолжение выполнения предыдущего задания, без переноса в новые папки. Она же смотрит в удаленный репозиторий <https://github.com/ArturPlrozhkov/vector-role>. Аналогично для необязательной части с `lighthouse`
- Ознакомительный запуск по `clickhouse-role` буду осуществлять из дефолтной папки, куда скачалась роль при установке `requirements`

из `~/ansible/roles/clickhouse`

- Разработка `Molecule` -сценариев буду делать локально на хосте.
- Контрольный прогон `tox` в контейнере `aragast/netology:latest`
- Установил модули:

```
1 sudo apt update && sudo apt install -y podman python3-pip
2 python3 -m pip install --break-system-packages ansible-core molecule
  molecule_docker molecule_podman
3 ansible-galaxy collection install containers.podman
```

поскольку работа ведется на виртуалке как на стенде, созданной специально для учебных задач, установку всех пакетов произвел в системную среду с ключом `--break-system-packages`, что недопустимо на основных рабочих станциях и живых машинах. Проверка настройки:

```
1 molecule --version
2 ansible --version
3 podman --version
4 docker --version
5 python3 -m pip show molecule molecule_docker molecule_podman
```

```
vboxuser@KVA:~/ansible/vector-role$ molecule --version
molecule 26.4.0 using python 3.12
ansible:2.21.1
podman:2.0.3 from molecule.podman requiring collections: containers.podman>=1.7.0 ansible.posix>=1.3.0
default:26.4.0 from molecule
docker:2.1.0 from molecule.docker requiring collections: community.docker>=3.0.2 ansible.posix>=1.4.0
vboxuser@KVA:~/ansible/vector-role$ ansible --version
ansible [core 2.21.1]
  config file = None
  configured module search path = ['/home/vboxuser/.ansible/plugins/modules', '/usr/share/ansible/plugins/modules']
  ansible python module location = /home/vboxuser/.local/lib/python3.12/site-packages/ansible
  ansible collection location = /home/vboxuser/.ansible/collections:/usr/share/ansible/collections
  executable location = /home/vboxuser/.local/bin/ansible
  python version = 3.12.3 (main, Mar 23 2026, 19:04:32) [GCC 13.3.0] (/usr/bin/python3)
  jinja version = 3.1.2
  pyyaml version = 6.0.1 (with libyaml v0.2.5)
vboxuser@KVA:~/ansible/vector-role$ podman --version
podman version 4.9.3
vboxuser@KVA:~/ansible/vector-role$ docker --version
Docker version 29.4.2, build 055a478
vboxuser@KVA:~/ansible/vector-role$ python3 -m pip show molecule molecule_docker molecule_podman
Name: molecule
Version: 26.4.0
Summary: Molecule aids in the development and testing of Ansible roles
Home-page:
Author:
Author-email: Ansible by Red Hat <info@ansible.com>
License:
Location: /home/vboxuser/.local/lib/python3.12/site-packages
Requires: ansible-compat, ansible-core, click, enrich, jinja2, jsonschema, packaging, pluggy, pyyaml, rich, wcmatch
Required-by: molecule-docker, molecule-podman
---
Name: molecule-docker
Version: 2.1.0
Summary: Molecule aids in the development and testing of Ansible roles
Home-page: https://github.com/ansible-community/molecule-docker
Author: Ansible by Red Hat
Author-email: info@ansible.com
License: MIT
Location: /home/vboxuser/.local/lib/python3.12/site-packages
Requires: docker, molecule, requests, selinux
Required-by:
vboxuser@KVA:~/ansible/vector-role$
Name: molecule-podman
Version: 2.0.3
Summary: Molecule aids in the development and testing of Ansible roles
Home-page: https://github.com/ansible-community/molecule-podman
Author: Ansible by Red Hat
Author-email: info@ansible.com
License: MIT
Location: /home/vboxuser/.local/lib/python3.12/site-packages
Requires: ansible-compat, molecule, selinux
Required-by:
```

Исходя из этой картины

```
vboxuser@KVA:~/ansible/vector-role$ molecule --version
molecule 26.4.0 using python 3.12
ansible:2.21.1
podman:2.0.3 from molecule.podman requiring collections: containers.podman>=1.7.0 ansible.posix>=1.3.0
default:26.4.0 from molecule
docker:2.1.0 from molecule.docker requiring collections: community.docker>=3.0.2 ansible.posix>=1.4.0
```

доустановил зависимости для `molecule-docker` и `molecule-podman` командой

```
1 ansible-galaxy collection install community.docker ansible.posix
```

Проверил установку командой

```
1 ansible-galaxy collection list | egrep  
'community.docker|containers.podman|ansible.posix'
```

```
vboxuser@KVA:~/ansible/vector-role$ ansible-galaxy collection list | egrep 'community.docker|containers.podman|ansible.posix'  
ansible.posix      2.2.0  
community.docker  5.2.1  
containers.podman  1.20.2
```

На этом настройка рабочего окружения окончена

Задание 1

Molecule

1. Запустите `molecule test -s ubuntu_xenial` (или с любым другим сценарием, не имеет значения) внутри корневой директории `clickhouse-role`, посмотрите на вывод команды. Данная команда может отработать с ошибками или не отработать вовсе, это нормально. Наша цель - посмотреть как другие в реальном мире используют молекулу И из чего может состоять сценарий тестирования.

Разбор решения задания

С учетом чтения переписки в учебном чате, выполнение данного задания не представляется возможным без адаптации `roles/clickhouse` со всем его наполнением для новых, последних установленных в систему версий необходимого для выполнения заданий ПО.

```
vboxuser@KVA:~/ansible/roles/clickhouse$ molecule test -s ubuntu_xenial 2>&1 | tee /tmp/molecule_clickhouse_ubuntu_xenial.log  
WARNING Driver docker does not provide a schema.  
ERROR Failed to validate /home/vboxuser/.ansible/roles/clickhouse/molecule/ubuntu_xenial/molecule.yml
```

2. Перейдите в каталог с ролью `vector-role` и создайте сценарий тестирования по умолчанию при помощи `molecule init scenario --driver-name docker`.
3. Добавьте несколько разных дистрибутивов (`oraclelinux:8`, `ubuntu:latest`) для инстансов и протестируйте роль, исправьте найденные ошибки, если они есть.

Разбор решения

- `Molecule` и отладку роли делаю на хосте в `~/ansible/vector-role`
- Иницирую сценарий

```
1 molecule init scenario default
```

```

• vboxuser@KVA:~/ansible/vector-role$ molecule init scenario default
INFO     default → init: Initializing new scenario default...

PLAY [Create a new molecule scenario] *****

TASK [Check if destination folder exists] *****
changed: [localhost]

TASK [Check if destination folder is empty] *****
ok: [localhost]

TASK [Fail if destination folder is not empty] *****
skipping: [localhost]

TASK [Expand templates] *****
changed: [localhost] => (item=molecule/default/verify.yml)
changed: [localhost] => (item=molecule/default/molecule.yml)
changed: [localhost] => (item=molecule/default/converge.yml)
changed: [localhost] => (item=molecule/default/destroy.yml)
changed: [localhost] => (item=molecule/default/create.yml)

PLAY RECAP *****
localhost                : ok=3    changed=2    unreachable=0    failed=0    skipped=1    rescued=0    ignored=0

INFO     default → init: Initialized scenario in /home/vboxuser/ansible/vector-role/molecule/default successfully.
• vboxuser@KVA:~/ansible/vector-role$

```

создан каркас:

```

• vboxuser@KVA:~/ansible/vector-role$ tree molecule
molecule
├── default
│   ├── converge.yml
│   ├── create.yml
│   ├── destroy.yml
│   ├── molecule.yml
│   └── verify.yml
└── 2 directories, 5 files

```

- в текущем каркасе исправляю файл `molecule/default/molecule.yml` на следующее содержимое:

```

1  ---
2      dependency:
3          name: shell
4          command: /bin/true
5
6      driver:
7          name: docker
8
9      platforms:
10         - name: ubuntu
11           image: ubuntu:latest
12           pre_build_image: true
13           command: sleep infinity
14
15         - name: oraclelinux
16           image: oraclelinux:8
17           pre_build_image: true
18           command: sleep infinity

```

```

19
20     provisioner:
21         name: ansible
22         env: ANSIBLE_ROLES_PATH:
23             "${MOLECULE_PROJECT_DIRECTORY}/../${MOLECULE_PROJECT_DIRECTORY}"
24         playbooks:
25             create: create.yml
26             prepare: prepare.yml
27             converge: converge.yml
28             destroy: destroy.yml
29             verify: verify.yml
30
31     scenario:
32         name: default
33         test_sequence:
34             - create
35             - prepare
36             - converge
37             - idempotence
38             - verify
39             - destroy
40
41     verifier:
42         name: ansible

```

С учетом особенностей развертывания `vector_role` из архива а не как `systemd_unit`, `dependency` -блок упрощен, потому что тестируемая роль не требует отдельного разрешения зависимостей `dependency resolution` через локальный `requirements.yml`:

```

1     dependency:
2         name: shell
3         command: /bin/true

```

Этот блок задает окружение, где будет развернута тестовая среда. Здесь явно указано `Molecule`: создавать инстансы нужно через `Docker` -контейнеры:

```

1     driver:
2         name: docker

```

Этот блок описывает, **какие именно тестовые инстансы** будут создаваться `Molecule`:

```

1     - name: ubuntu

```

```

2     image: ubuntu:latest
3
4
5     - name: oraclelinux
6       image: oraclelinux:8

```

`pre_build_image: true` - использовать образ как уже готовый и не пытаться собирать его через `Dockerfile.j2`.

`command: sleep infinity` - команда, чтобы контейнер не потух сразу после запуска.

Блок `provisioner` говорит, чем применять конфигурацию к тестовым инстансам.

Используется обычный Ansible и явно указываю на то, какие `playbook` использовать в `vector-role`: `create: create.yml` - создает контейнер из предустановленного образа (`pre_build_image: true`). `env:`

`ANSIBLE_ROLES_PATH:`

`"${MOLECULE_PROJECT_DIRECTORY}/../${MOLECULE_PROJECT_DIRECTORY}"` - переменная среды окружения явно указывает путь до роли (иначе ищет в дефолтных путях, игнорируя папку разработки). `playbooks.prepare:`

`prepare.yml` - перед применением роли нужно выполнить отдельный `bootstrap-playbook`. Это нужно потому, что на `ubuntu:latest` и `oraclelinux:8` может не быть `Python`, а без него обычные модули `Ansible` не смогут работать.

`converge.yml` будет запускать саму роль `vector-role` а `verify.yml` будет проверять, что бинарник `Vector` появился, конфиг создан и `vector`

`validate` проходит успешно:

```

1  provisioner:
2      name: ansible
3      env: ANSIBLE_ROLES_PATH:
4           "${MOLECULE_PROJECT_DIRECTORY}/../${MOLECULE_PROJECT_DIRECTORY}"
5      playbooks:
6          create: create.yml
7          prepare: prepare.yml
8          converge: converge.yml
9          destroy: destroy.yml
          verify: verify.yml

```

Имя сценария, команды `molecule test` и `molecule converge` можно запускать без явного указания имени сценария:

```

1  scenario:
2      name: default

```

`test_sequence` - последовательность этапов, которые выполнит `molecule test`

```
1  test_sequence:
2    - create
3    - prepere
4    - converge
5    - idempotence
6    - verify
7    - destroy
```

- `create` - `Molecule` создает контейнеры `ubuntu` и `oraclelinux`;
- `prepere` - наполнение контейнеров необходимым базовым набором ПО из соответствующего файла;
- `converge` - `Ansible` применяет `vector-role` на оба контейнера;
- `idempotence` - `Molecule` повторно запускает роль и проверяет, что второй прогон не вносит лишних изменений;
- `verify` - запускаются проверки результата;
- `destroy` - контейнеры удаляются;
- пока нет `cleanup.yml`, `side_effect.yml` для упрощения и стабильности

Создал и заполнил файл `prepare.yml` (пока только для отработки управления инстансами через `ansible`, позднее наполнение нужными утилитами):

```
1  ---
2  - name: Prepare
3    hosts: all
4    gather_facts: false
5    tasks:
6      - name: Install Python on Ubuntu
7        ansible.builtin.raw: |
8          test -e /usr/bin/python3 || (apt-get update && apt-get
install -y python3)
9        changed_when: false
10       failed_when: false
11       when: inventory_hostname == "ubuntu"
12
13      - name: Install Python 3.9 on Oracle Linux
14        ansible.builtin.raw: |
15          test -e /usr/bin/python3.9 || (dnf install -y python39)
16        changed_when: false
17        failed_when: false
18        when: inventory_hostname == "oraclelinux"
19
20      - name: Reset connection
```


логика происходящего:

- не собирать `facts` в начале;
- через `raw` определить, есть ли `Python` (https://docs.ansible.com/projects/ansible/latest/collections/ansible/builtin/raw_module.html);
- если `Python` нет, то установить его;
- после этого сбросить `connection`, чтобы `Ansible` начал использовать уже доступный `Python`

Пояснения к содержимому файла:

- `hosts: all` - применимо для обоих контейнеров, потому что оба являются целевыми тестовыми хостами сценария;
- `gather_facts: false` - `Facts` выключены, потому что до установки `Python` их сбор не работает;
- `test -e /usr/bin/python3 || (apt-get update && apt-get install -y python3)` - если `python3` уже есть, команда завершается без установки. Если нет, то после обновления ставится Python 3 (для `ubuntu`);
- `test -e /usr/bin/python3 || dnf install -y python39` - аналогично для **Oracle Linux**;
- `reset_connection` - после установки `Python`, `Ansible` должен переподключиться к контейнеру, чтобы дальше уже использовать свои собственные инструменты.

Заменяю содержимое `molecule/default/converge.yml` на это:

```
1 ---
2 - name: Converge
3   hosts: all
4   gather_facts: true
5   tasks:
6     - name: Include vector-role
7       ansible.builtin.include_role:
8         name: vector-role
```

- `hosts: all` - проверка на обоих контейнерах, которые описаны в `platforms`: и на `Ubuntu`, и на `Oracle Linux`;
- `gather_facts: true` - включено после `prepare.yml`, `Python` на целевых контейнерах должен быть установлен (в `vector-role` используются `facts` -

например `ansible_facts['user_dir']`)

- `ansible.builtin.include_role` - подключение тестируемой роли

Заменял содержимое `verify.yml` на это:

```
1  ---
2  - name: Verify
3    hosts: all
4    gather_facts: true
5    tasks:
6      - name: Check vector install directory
7        ansible.builtin.stat:
8          path: "{{ ansible_facts['user_dir'] }}/vector"
9          register: vector_dir
10
11     - name: Check vector binary
12       ansible.builtin.stat:
13         path: "{{ ansible_facts['user_dir'] }}/vector/bin/vector"
14         register: vector_bin
15
16     - name: Check vector config
17       ansible.builtin.stat:
18         path: "{{ ansible_facts['user_dir']
19 }}/vector/config/vector.yml"
20       register: vector_cfg
21
22     - name: Validate vector config
23       ansible.builtin.command:
24         cmd: "{{ ansible_facts['user_dir'] }}/vector/bin/vector
25 validate {{ ansible_facts['user_dir'] }}/vector/config/vector.yml"
26       register: vector_validate
27       changed_when: false
28
29     - name: Assert vector role results
30       ansible.builtin.assert:
31         that:
32           - vector_dir.stat.exists
33           - vector_dir.stat.isdir
34           - vector_bin.stat.exists
35           - vector_bin.stat.isreg
36           - vector_cfg.stat.exists
37           - vector_cfg.stat.isreg
38           - vector_validate.rc == 0
```

- `hosts: all` - проверка на обоих контейнерах - на Ubuntu, и на Oracle Linux;

- `gather_facts: true` - сбор фактов нужен, потому что роль ставит Vector в `ansible_facts['user_dir'] {{/vector}}`;
- `stat` - способ проверить, что директории и файлы реально созданы (<https://oneuptime.com/blog/post/2026-02-21-how-to-write-molecule-verify-tests-with-ansible/view>);
- `assert` - собирает все проверки в один понятный `verification step` (<https://oneuptime.com/blog/post/2026-02-21-ansible-assert-based-tests/view>)

Тестирую по частям:

```
1 molecule create
```

Сходу ошибка - из-за проверки имени роли по правилам `Galaxy`. `Molecule` через `ansible-compat` вычислила полное имя `ArturPlrozhkov.vector-role` и сочла его некорректным, потому что `role name` с дефисом не соответствует текущим требованиям

(<https://docs.ansible.com/projects/molecule/configuration/#variable-substitution>, <https://stackoverflow.com/questions/73391894/ansible-naming-rules-for-roles-and-collections-dash-underscore>).

Переименовываю роль в локальных хостовых папках разработки с последующим пушем на гитхаб.

Файл `meta/main.yml`

```
1 ---
2 galaxy_info:
3     author: ArturPlrozhkov
4     namespace: arturplrozhkov
5     role_name: vector_role
6     description: Ansible role for Vector
7     license: MIT
8     min_ansible_version: "2.10"
9     galaxy_tags:
10         - vector
11
12     dependencies: []
```

- `namespace: arturplrozhkov` - формат, который совместим с Galaxy naming rules, без дефисов и прочих спорных символов;
- `role_name: vector_role` - вместо вычисленного `vector-role` задаем валидное имя `vector_role` с подчеркиванием (https://docs.ansible.com/ansible-galaxy/latest/contributing/creating_role.html)

Исправил имя роли в `/vector-role/molecule/default/converge.yml`

```
1 ----
2 - name: Converge
3   hosts: all
4   gather_facts: true
5   tasks:
6     - name: Include role under test
7       ansible.builtin.include_role:
8         name: "{{ lookup('env', 'MOLECULE_PROJECT_DIRECTORY') }}"
```

Указал явно путь с ролью

Запустил проверку `molecule create`

```
vboxuser@KVA:~/ansible/vector-role$ molecule create
WARNING Driver docker does not provide a schema.
WARNING Driver docker does not provide a schema.
INFO default → discovery: scenario test matrix: dependency, create, prepare
INFO default → prerun: Performing prerun with role_name_check=0...
INFO default → dependency: Executing
INFO default → dependency: Dependency completed successfully.
INFO default → dependency: Executed: Successful
INFO default → create: Executing
INFO Sanity checks: 'docker'

PLAY [Create] *****

TASK [Populate instance config dict] *****
skipping: [localhost]

TASK [Convert instance config dict to a list] *****
skipping: [localhost]

TASK [Dump instance config] *****
skipping: [localhost]

PLAY RECAP *****
localhost : ok=0 changed=0 unreachable=0 failed=0 skipped=3 rescued=0 ignored=0

INFO default → create: Executed: Successful
INFO default → prepare: Executing
WARNING default → prepare: Executed: Missing playbook (Remove from create_sequence to suppress)
WARNING Molecule executed 1 scenario (1 missing files)
```

контейнеры **не создаются**, потому что текущий `create.yml` это незаполненный шаблон-заглушка

`create.yml` и `destroy.yml` реализую

через `community.docker.docker_container` (установлен)

(<https://docs.ansible.com/projects/molecule/examples/docker/>)

```
vboxuser@KVA:~/ansible/vector-role$ ansible-galaxy collection list | grep community.docker
community.docker 5.2.1
```

Заменяю содержимое `create.yml` на это:

```
1 ----
2 - name: Create
3   hosts: localhost
4   connection: local
5   gather_facts: false
6   vars:
7     molecule_inventory:
8       all:
```

```

9         children:
10             molecule:
11                 hosts: {}
12 tasks:
13     - name: Create a container
14       community.docker.docker_container:
15         name: "{{ item.name }}"
16         image: "{{ item.image }}"
17         state: started
18         command: "{{ item.command }}"
19         tty: true
20         detach: true
21         privileged: "{{ item.privileged | default(false) }}"
22       register: result
23       loop: "{{ molecule_yaml.platforms }}"
24
25     - name: Fail if container is not running
26       ansible.builtin.assert:
27         that:
28             - item.container is defined
29             - item.container.State is defined
30             - item.container.State.Running
31         fail_msg: "Container {{ item.item.name }} is not running"
32         loop: "{{ result.results }}"
33       loop_control:
34         label: "{{ item.item.name }}"
35
36     - name: Add container to molecule inventory
37       vars:
38         inventory_partial_yaml: |
39             all:
40                 children:
41                     molecule:
42                         hosts:
43                             "{{ item.name }}":
44                                 ansible_connection: community.docker.docker
45                                 ansible_python_interpreter: "{{
46 '/usr/bin/python3.9' if item.name == 'oraclelinux' else
47 '/usr/bin/python3' }}"
48         ansible.builtin.set_fact:
49             molecule_inventory: >-
50                 {{ molecule_inventory | combine(inventory_partial_yaml |
51 from_yaml, recursive=True) }}
52         loop: "{{ molecule_yaml.platforms }}"
53       loop_control:
54         label: "{{ item.name }}"

```

```

52
53     - name: Dump molecule inventory
54       ansible.builtin.copy:
55         content: "{{ molecule_inventory | to_nice_yaml }}"
56         dest: "{{ molecule_ephemeral_directory
57             }}/inventory/molecule_inventory.yml"
58         mode: "0600"
59
60     - name: Refresh inventory
61       ansible.builtin.meta: refresh_inventory

```

- `hosts: localhost` - этот playbook не управляет целевыми контейнерами напрямую как `managed nodes`. Он выполняется локально и через Docker API **создает** тестовые инстансы.
- переменные `vars`: создаем переменную `molecule_inventory` и кладем в нее начальную структуру `inventory`, где есть группа `all`, внутри нее дочерняя группа `molecule` (далее playbook'и будут удобно запускаться по `hosts: molecule`), а список хостов пока пустой
- `gather_facts: false` - не собираем факты до выполнения `prepare`
- `community.docker.docker_container` - это основной модуль, который поднимает контейнеры `ubuntu` и `oraclelinux` по данным из `platforms`.
- `state: started` - контейнер должен быть запущен в фоне (`detach: true`)
- `privileged: "{{ item.privileged | default(false) }}"` - если у платформы есть `privileged`, берется оно; если нет, подставляется `false`.
- `command: "{{ item.command }}"` - команда внутри контейнера тоже подставляется из `platforms` (в нашем случае `sleep infinity`)
- `register: result` - `register` сохраняет результат выполнения задачи в переменную. При использовании `loop` там будет не один результат, а список результатов всех итераций (`result.results`)
- `molecule_yaml.platforms` - используются уже описанные платформы из `molecule.yml`
- `molecule_inventory` - после создания контейнеров Molecule должна понять, **как к ним подключаться**. Для Docker в `ansible-native` сценарии это делается через `inventory` с `ansible_connection: community.docker.docker`.
- `molecule_ephemeral_directory` - `inventory` пишется во временный каталог сценария, откуда Molecule затем берет динамически сгенерированные хосты для `prepare`, `converge` и `verify`
- `loop: "{{ molecule_yaml.platforms }}"` - повторяет эту задачу для каждого элемента списка `platforms` из `molecule.yml`, текущий элемент цикла в каждой итерации доступен через переменную `item`

- в конструкции

```

1  name: "{{ item.name }}"
2  image: "{{ item.image }}"
3  command: "{{ item.command }}"
4  privileged: "{{ item.privileged | default(false) }}"

```

поля `name`, поле `image`, поле `command` и тд берутся у текущего элемента цикла

- в конструкции:

```

1      - name: Add container to molecule inventory
2        vars:
3          inventory_partial_yaml: |
4            all:
5              children:
6                molecule:
7                  hosts:
8                    "{{ item.name }}":
9                      ansible_connection: community.docker.docker
10                     ansible_python_interpreter: "{{
11                       '/usr/bin/python3.9' if item.name == 'oraclelinux' else
12                       '/usr/bin/python3' }}"
13                     ansible.builtin.set_fact:
14                       molecule_inventory: >-
15                         {{ molecule_inventory | combine(inventory_partial_yaml |
16                           from_yaml, recursive=True) }}
17                     loop: "{{ molecule_yaml.platforms }}"
18                     loop_control:
19                       label: "{{ item.name }}"

```

хранится строковое значение поля `molecule_inventory`, а внутри этой строки `Jinja` вычисляет выражение, подставляет его в существующий `molecule_inventory` и возвращает готовый словарь. `recursive=True` означает рекурсивное объединение вложенных словарей, чтобы новый хост добавлялся внутрь уже существующей структуры, и не затирал ее целиком.

`ansible_python_interpreter: "{{ '/usr/bin/python3.9' if item.name == 'oraclelinux' else '/usr/bin/python3' }}"` - для oracle linux явно указываю интерпретатор Python

- в конструкции:

```

1  - name: Dump molecule inventory
2    ansible.builtin.copy:

```

```

3     content: "{{ molecule_inventory | to_nice_yaml }}"
4     dest: "{{ molecule_ephemeral_directory
    }}/inventory/molecule_inventory.yml"
5     mode: "0600"

```

- `molecule_inventory` - структура данных в памяти (словарь Ansible);
- `to_yaml` или `to_nice_yaml` - превратить ее в текст YAML;
- `copy` - записать этот текст в файл `inventory`.
- `dest: "{{ molecule_ephemeral_directory }}/inventory/molecule_inventory.yml"` - сохранить `inventory` во временный служебный каталог Molecule, откуда следующие шаги сценария потом его прочитают

```

1     - name: Refresh inventory
2       ansible.builtin.meta: refresh_inventory

```

говорит перечитать `inventory` после того, как в него записали новый файл с хостами.

В итоге `create.yml` создает контейнеры в цикле по `platforms`, собирает по ним динамический `inventory` и сохраняет его в файл, чтобы этапы `prepare`, `converge` и `verify` могли обращаться к группе `molecule` как к обычным `managed hosts`.

Заменяю содержимое `destroy.yml` на это:

```

1  ---
2  - name: Destroy molecule containers
3    hosts: molecule
4    gather_facts: false
5    tasks:
6      - name: Stop and remove container
7        delegate_to: localhost
8        community.docker.docker_container:
9          name: "{{ inventory_hostname }}"
10         state: absent
11         auto_remove: true
12
13
14 - name: Remove dynamic molecule inventory
15   hosts: localhost
16   gather_facts: false
17   tasks:
18     - name: Remove dynamic inventory file

```



```
19     ansible.builtin.file:
20         path: "{{ molecule_ephemeral_directory
21             }}/inventory/molecule_inventory.yml"
22         state: absent
```

В нем:

```
1  - name: Destroy molecule containers
2    hosts: molecule
3    gather_facts: false
```

- `hosts: molecule` означает, что play запускается для всех хостов из группы `molecule`, которые собраны в `create.yml`. Для этого создан динамический `inventory`.
- `gather_facts: false` - facts не нужны, нужно просто удалить контейнеры. Play итерируется по хостам группы `molecule`, но удаление контейнеров реально происходит не внутри контейнера, а на локальной машине через Docker API.

```
1  - name: Stop and remove container
2    delegate_to: localhost
3    community.docker.docker_container:
4        name: "{{ inventory_hostname }}"
5        state: absent
6        auto_remove: true
```

- `delegate_to: localhost` - означает, что несмотря на то, что `play` идет по хостам `molecule`, именно эту задачу выполняй на `localhost`. Контейнер не должен удалять себя изнутри, это операция Docker на хостовой машине, а значит команду надо делегировать туда, где запущен Docker daemon.
- `state: absent` - контейнер должен отсутствовать. Если он существует - удалить.
- `auto_remove: true` - Docker должен автоматически удалить контейнер после остановки.

```
1  - name: Remove dynamic molecule inventory
2    hosts: localhost
3    gather_facts: false
```

После удаления контейнеров остается файл динамического `inventory`, созданный в `create.yml`.

Этот `play` уже работает просто на `localhost`, потому что контейнеров уже может не быть.

```
1 - name: Remove dynamic inventory file
2   ansible.builtin.file:
3     path: "{{ molecule_ephemeral_directory
4           }}/inventory/molecule_inventory.yml"
5     state: absent
```

Удаление inventory-файла.

- `ansible.builtin.file` - это стандартный модуль для управления файлами и директориями.
 - `path: "{{ molecule_ephemeral_directory }}/inventory/molecule_inventory.yml"` - временный inventory-файл
 - `molecule_ephemeral_directory` - служебный временный каталог текущего сценария Molecule.
 - `state: absent` - удалить файл, если он существует.
- `destroy.yml` делает два последовательных шага: пройти по всем контейнерам из группы `molecule` и удалить каждый контейнер через Docker на `localhost`. Удалить временный файл `inventory`, чтобы сценарий не оставлял после себя старое состояние. Это необходимо для того, чтобы следующий запуск стартовал с нуля, а не использовал остатки прошлого запуска.

Запускаю снова команду `molecule create`. До этого необходимо очистить состояние: `molecule destroy` и `molecule reset`

```
vboxuser@KVA:~/ansible/vector-roles$ molecule list
WARNING Driver docker does not provide a schema.
INFO default + list: Executing
INFO default + list: Executed: Successful
```

Instance Name	Driver Name	Provisioner Name	Scenario Name	Created	Converged
ubuntu	docker	ansible	default	true	false
oraclelinux	docker	ansible	default	true	false

```
vboxuser@KVA:~/ansible/vector-roles$ docker ps -a
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS          NAMES
51155512101a   oraclelinux:8 "sleep infinity"        45 minutes ago Up 45 minutes                   oraclelinux
794faf5b9359   ubuntu:latest  "sleep infinity"        45 minutes ago Up 45 minutes                   ubuntu
vboxuser@KVA:~/ansible/vector-roles$
```

Запускаю команду `molecule converge`. Ошибки - версия Python на Oracle_Linux в версии ниже 3.9+, минимально необходимой для Ansible-core 2.21 и поиск роли в дефолтных путях но не в текущей папке разработки. Добавил изменения в виде версии Python и его интерпретатора явно а также переменную среды окружения `env: ANSIBLE_ROLES_PATH:`

`"${MOLECULE_PROJECT_DIRECTORY}/../${MOLECULE_PROJECT_DIRECTORY}"` в файлы `prepare.yml` и `molecule/default/molecule.yml` и `converge.yml`

Запустил по новой предварительно все очистив:

```

1 molecule destroy
2 molecule reset
3 docker rm -f ubuntu oraclelinux 2>/dev/null || true
4 molecule create
5 molecule prepare
6 molecule converge

```

```

• vboxuser@KVA:~/ansible/vector-role$ molecule list
WARNING Driver docker does not provide a schema.
INFO default → list: Executing
INFO default → list: Executed: Successful

```

Instance Name	Driver Name	Provisioner Name	Scenario Name	Created	Converged
ubuntu	docker	ansible	default	true	false
oraclelinux	docker	ansible	default	true	false

```

• vboxuser@KVA:~/ansible/vector-role$ docker ps -a
CONTAINER ID   IMAGE             COMMAND                  CREATED        STATUS        PORTS   NAMES
39e9a6ea70c3   oraclelinux:8     "sleep infinity"        5 minutes ago Up 5 minutes   ubuntu
6c4272aaba68   ubuntu:latest     "sleep infinity"        5 minutes ago Up 5 minutes   oraclelinux
• vboxuser@KVA:~/ansible/vector-role$

```

В момент `molecule converge` посыпались ошибки: `get_url` на Oracle Linux падает по таймауту, а шаблон `vector.yml.j2` содержит только `data_dir`, тогда как `vector validate` требует хотя бы один `source` и один `sink` (<https://vector.dev/docs/administration/validating/>, <https://github.com/vectordev/vector/issues/23463>). У `get_url` timeout по умолчанию 10 секунд, и этого может не хватать для скачивания архива с GitHub из контейнера (<https://www.ansiblebyexample.com/articles/ansible-download-file-from-url-get-url-uri>).

Добавляю в `defaults/main.yml`

```

1 ---
2 vector_version: "0.37.1"
3 vector_install_dir: "{{ ansible_facts['user_dir'] }}/vector"
4
5 vector_sources:
6     demo_stdin:
7         type: stdin
8
9 vector_sinks:
10     demo_console:
11         type: console
12         inputs:
13             - demo_stdin
14         encoding:
15             codec: text

```

Добавляю в `templates/vector.yml.j2`

```

1 data_dir: "{{ vector_install_dir }}/data"
2
3 sources:
4 {{ vector_sources | to_nice_yaml(indent=2) | indent(2, true) }}
5
6 sinks:
7 {{ vector_sinks | to_nice_yaml(indent=2) | indent(2, true) }}

```

Добавляю в `tasks/main.yml`

```

1 ---
2 - name: Create vector install directory
3   ansible.builtin.file:
4     path: "{{ vector_install_dir }}"
5     state: directory
6     mode: "0755"
7   when: not ansible_check_mode
8
9 - name: Get Vector distrib
10  ansible.builtin.get_url:
11    url:
12      "https://github.com/vectordev/vector/releases/download/v{{
13      vector_version }}/vector-{{ vector_version }}-x86_64-unknown-linux-
14      musl.tar.gz"
15    dest: "/tmp/vector-{{ vector_version }}.tar.gz"
16    mode: "0644"
17    timeout: 60
18    register: vector_download
19    retries: 3
20    delay: 5
21    until: vector_download is succeeded
22    when: not ansible_check_mode
23
24 - name: Unarchive Vector
25   ansible.builtin.unarchive:
26     src: "/tmp/vector-{{ vector_version }}.tar.gz"
27     dest: "{{ vector_install_dir }}"
28     remote_src: true
29     extra_opts:
30       - "--strip-components=2"
31     creates: "{{ vector_install_dir }}/bin/vector"
32   when: not ansible_check_mode
33
34 - name: Create vector data directory
35   ansible.builtin.file:

```

```

33     path: "{{ vector_install_dir }}/data"
34     state: directory
35     mode: "0755"
36 when: not ansible_check_mode
37
38 - name: Create vector config directory
39   ansible.builtin.file:
40     path: "{{ vector_install_dir }}/config"
41     state: directory
42     mode: "0755"
43   when: not ansible_check_mode
44
45 - name: Deploy vector config
46   ansible.builtin.template:
47     src: vector.yml.j2
48     dest: "{{ vector_install_dir }}/config/vector.yml"
49     mode: "0644"
50   notify: Validate vector config
51   when: not ansible_check_mode

```

- timeout 60 вместо дефолтных 10 секунд снижает риск случайного падения на GitHub download.
- `retries` и `until` добавляют устойчивость к сетевым лагам.
- `data_dir` создаем явно, раз он есть в конфиге.

Добавил в файл `prepare.yml` установку ПО для vector

```

1  ---
2  - name: Prepare
3    hosts: all
4    gather_facts: false
5    tasks:
6      - name: Bootstrap Python on Ubuntu
7        ansible.builtin.raw: |
8          test -e /usr/bin/python3 || (apt-get update && apt-get
install -y python3)
9          changed_when: false
10         failed_when: false
11         when: inventory_hostname == "ubuntu"
12
13      - name: Bootstrap Python 3.9 on Oracle Linux
14        ansible.builtin.raw: |
15          test -e /usr/bin/python3.9 || (dnf install -y python39)
16          changed_when: false
17          failed_when: false

```

```

18     when: inventory_hostname == "oraclelinux"
19
20     - name: Reset connection
21       ansible.builtin.meta: reset_connection
22
23     - name: Gather facts after Python bootstrap
24       ansible.builtin.setup:
25
26     - name: Install utility packages on Ubuntu
27       ansible.builtin.apt:
28         name:
29           - ca-certificates
30           - tar
31           - gzip
32           - unzip
33         state: present
34         update_cache: true
35       when: inventory_hostname == "ubuntu"
36
37     - name: Install utility packages on Oracle Linux
38       ansible.builtin.dnf:
39         name:
40           - ca-certificates
41           - tar
42           - gzip
43           - unzip
44         state: present
45       when: inventory_hostname == "oraclelinux"

```

Добавил в файл `ansible/vector-role/handlers/main.yml` строку `--no-environment`

```

1  ---
2  - name: Validate vector config
3    ansible.builtin.command:
4      cmd: "{{ vector_install_dir }}/bin/vector validate --no-
5            environment {{ vector_install_dir }}/config/vector.yml"
6    changed_when: false

```

чтобы валидация конфига вектора не заваливалась из-за особенностей окружения контейнера, которые к самой роли отношения не имеют.

Запускаю `molecule test`

```

TASK [Add container to molecule inventory] *****
ok: [localhost] => (item=ubuntu)
ok: [localhost] => (item=oraclelinux)

TASK [Dump molecule inventory] *****
changed: [localhost]

TASK [Refresh inventory] *****

PLAY RECAP *****
localhost                : ok=4    changed=2    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0

INFO     default -> create: Executed: Successful
INFO     default -> prepare: Executing

PLAY [Prepare] *****

TASK [Bootstrap Python on Ubuntu] *****
skipping: [oraclelinux]
ok: [ubuntu]

TASK [Bootstrap Python 3.9 on Oracle Linux] *****
skipping: [ubuntu]
ok: [oraclelinux]

TASK [Reset connection] *****

TASK [Reset connection] *****

TASK [Gather facts after Python bootstrap] *****
ok: [ubuntu]
ok: [oraclelinux]

TASK [Install utility packages on Ubuntu] *****
skipping: [oraclelinux]
changed: [ubuntu]

TASK [Install utility packages on Oracle Linux] *****
skipping: [ubuntu]
changed: [oraclelinux]

```

```

PLAY RECAP *****
oraclelinux              : ok=3    changed=1    unreachable=0    failed=0    skipped=2    rescued=0    ignored=0
ubuntu                  : ok=3    changed=1    unreachable=0    failed=0    skipped=2    rescued=0    ignored=0

INFO     default -> prepare: Executed: Successful
INFO     default -> converge: Executing

PLAY [Converge] *****

TASK [Gathering Facts] *****
ok: [ubuntu]
ok: [oraclelinux]

TASK [Include role under test] *****
included: /home/vboxuser/ansible/vector-role for oraclelinux, ubuntu

TASK [/home/vboxuser/ansible/vector-role : Create vector install directory] ****
changed: [oraclelinux]
changed: [ubuntu]

TASK [/home/vboxuser/ansible/vector-role : Get Vector distrib] *****
changed: [oraclelinux]
changed: [ubuntu]

TASK [/home/vboxuser/ansible/vector-role : Unarchive Vector] *****
changed: [ubuntu]
changed: [oraclelinux]

TASK [/home/vboxuser/ansible/vector-role : Create vector data directory] *****
changed: [oraclelinux]
changed: [ubuntu]

TASK [/home/vboxuser/ansible/vector-role : Create vector config directory] *****
ok: [oraclelinux]
ok: [ubuntu]

TASK [/home/vboxuser/ansible/vector-role : Deploy vector config] *****
changed: [oraclelinux]
changed: [ubuntu]

RUNNING HANDLER [/home/vboxuser/ansible/vector-role : Validate vector config] ***
ok: [oraclelinux]
ok: [ubuntu]

```

```

PLAY RECAP *****
oraclelinux              : ok=9    changed=5    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
ubuntu                  : ok=9    changed=5    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0

INFO     default -> converge: Executed: Successful
INFO     default -> idempotence: Executing

PLAY [Converge] *****

TASK [Gathering Facts] *****
ok: [ubuntu]
ok: [oraclelinux]

TASK [Include role under test] *****
included: /home/vboxuser/ansible/vector-role for oraclelinux, ubuntu

TASK [/home/vboxuser/ansible/vector-role : Create vector install directory] ****
ok: [oraclelinux]
ok: [ubuntu]

TASK [/home/vboxuser/ansible/vector-role : Get Vector distrib] *****
ok: [oraclelinux]
ok: [ubuntu]

TASK [/home/vboxuser/ansible/vector-role : Unarchive Vector] *****
skipping: [oraclelinux]
skipping: [ubuntu]

TASK [/home/vboxuser/ansible/vector-role : Create vector data directory] *****
ok: [oraclelinux]
ok: [ubuntu]

TASK [/home/vboxuser/ansible/vector-role : Create vector config directory] *****
ok: [oraclelinux]
ok: [ubuntu]

TASK [/home/vboxuser/ansible/vector-role : Deploy vector config] *****
ok: [oraclelinux]
ok: [ubuntu]

PLAY RECAP *****
oraclelinux              : ok=7    changed=0    unreachable=0    failed=0    skipped=1    rescued=0    ignored=0
ubuntu                  : ok=7    changed=0    unreachable=0    failed=0    skipped=1    rescued=0    ignored=0

```

```

INFO default → idempotence: Executed: Successful
INFO default → verify: Executing

PLAY [Verify] *****
TASK [Gathering Facts] *****
ok: [ubuntu]
ok: [oraclelinux]

TASK [Check vector install directory] *****
ok: [oraclelinux]
ok: [ubuntu]

TASK [Check vector binary] *****
ok: [oraclelinux]
ok: [ubuntu]

TASK [Check vector config] *****
ok: [oraclelinux]
ok: [ubuntu]

TASK [Validate vector config] *****
ok: [oraclelinux]
ok: [ubuntu]

TASK [Assert vector role results] *****
ok: [oraclelinux] => {
  "changed": false,
  "msg": "All assertions passed"
}
ok: [ubuntu] => {
  "changed": false,
  "msg": "All assertions passed"
}

PLAY RECAP *****
oraclelinux : ok=6  changed=0  unreachable=0  failed=0  skipped=0  rescued=0  ignored=0
ubuntu      : ok=6  changed=0  unreachable=0  failed=0  skipped=0  rescued=0  ignored=0

INFO default → verify: Executed: Successful
INFO default → destroy: Executing

PLAY [Destroy molecule containers] *****
TASK [Stop and remove container] *****
changed: [ubuntu -> localhost]
changed: [oraclelinux -> localhost]

PLAY [Remove dynamic molecule inventory] *****
TASK [Remove dynamic inventory file] *****
changed: [localhost]

PLAY RECAP *****
localhost : ok=1  changed=1  unreachable=0  failed=0  skipped=0  rescued=0  ignored=0
oraclelinux : ok=1  changed=1  unreachable=0  failed=0  skipped=0  rescued=0  ignored=0
ubuntu      : ok=1  changed=1  unreachable=0  failed=0  skipped=0  rescued=0  ignored=0

INFO default → destroy: Executed: Successful
INFO default → scenario: Pruning extra files from scenario ephemeral directory
INFO Molecule executed 1 scenario (1 successful)

```

Тестирование прошло успешно. Второй раз тоже.

Запустил и затегировал в гитхаб изменения:

```

1  git status
2  git remote -v
3  git add -A
4  git status
5  git commit -m "Create and Fix Molecule scenario and verify Vector
   role"
6  git push origin main
7  git tag -a 0.0.2 -m "Working Molecule scenario for vector role"
8  git push origin 0.0.2

```

Tox

- Скачал и положил в рабочую директорию проекта файлы. С учетом ошибок коллег в чате подкорректировал `tox-requirements.txt` так:

```

1  ansible-core>=2.15,<2.18
2  molecule>=24.0.0
3  molecule-plugins[podman]>=23.0.0
4  jmespath
5  lxml

```

- `ansible-core` ставлю явно;

- `molecule-plugins[podman]` использую вместо старого `molecule_podman` ;
- `jmespath` и `lxml` оставляем как безопасные зависимости;
- `selinux` пока убрал, чтобы не тащить лишний потенциальный источник проблем.

аналогично с `tox.ini`

```

1  [tox]
2  minversion = 4.0
3  envlist = molecule
4  skipsdist = true
5
6  [testenv]
7  basepython = python3
8  passenv = *
9  deps =
10     -r tox-requirements.txt
11  commands =
12     {posargs:molecule test -s podman --destroy always}

```

- один тестовый env;
- один облегченный scenario;
- без привязки к Python 3.6 и древним Ansible 2.10/3.0.
- Создал директорию `ansible/vector-role/molecule/podman` и в ней файл `molecule.yml`

```

1  ---
2  dependency:
3      name: galaxy
4
5  driver:
6      name: podman
7
8  platforms:
9      - name: vector-podman
10        image: docker.io/python:3.11-slim
11        pre_build_image: true
12        command: sleep infinity
13
14  provisioner:
15      name: ansible
16      inventory:
17          hosts:
18              all:

```

```

19         hosts:
20             vector-podman:
21                 ansible_connection: containers.podman.podman
22     playbooks:
23         create: create.yml
24         destroy: destroy.yml
25
26     scenario:
27         name: podman
28         test_sequence:
29             - create
30             - converge
31             - idempotence
32             - verify
33             - destroy
34
35     verifier:
36         name: ansible

```

- `driver: name: podman` - это отдельный podman-сценарий Molecule.
- `python:3.11-slim` уменьшает шанс, что снова придётся bootstrap-ить Python.
- `sleep infinity` удерживает контейнер запущенным
- явно указал `inventory`, какие playbooks запускать на этапах `create` и `destroy`, - что подключение к контейнеру идет через `containers.podman.podman`.
- создал файл `converge.yml`

```

1  ---
2  - name: Converge
3    hosts: all
4    gather_facts: true
5    roles:
6      - role: /home/vboxuser/ansible/vector-role

```

из-за постоянных ошибок поиска роли пришлось захардкодить абсолютный путь к папке разработки. Так как в podman-сценарии выбран образ `python:3.11-slim`, Python внутри уже есть, поэтому можно сразу включать `gather_facts: true`

- Создал файл `verify.yml`

```

1  ---
2  - name: Verify
3    hosts: all
4    gather_facts: true

```

```

5 vars:
6   vector_install_dir: "{{ ansible_facts['user_dir'] }}/vector"
7
8 tasks:
9   - name: Check vector install directory
10     ansible.builtin.stat:
11       path: "{{ vector_install_dir }}"
12     register: vector_dir
13
14   - name: Check vector binary
15     ansible.builtin.stat:
16       path: "{{ vector_install_dir }}/bin/vector"
17     register: vector_bin
18
19   - name: Check vector config
20     ansible.builtin.stat:
21       path: "{{ vector_install_dir }}/config/vector.yml"
22     register: vector_cfg
23
24   - name: Validate vector config
25     ansible.builtin.command:
26       cmd: "{{ vector_install_dir }}/bin/vector validate --no-
environment {{ vector_install_dir }}/config/vector.yml"
27     register: vector_validate
28     changed_when: false
29
30   - name: Assert vector role results
31     ansible.builtin.assert:
32       that:
33         - vector_dir.stat.exists
34         - vector_dir.stat.isdir
35         - vector_bin.stat.exists
36         - vector_bin.stat.isreg
37         - vector_cfg.stat.exists
38         - vector_cfg.stat.isreg
39         - vector_validate.rc == 0

```

по подобию того, который был в прошлом задании

- создал файл `create.yml`

```

1 ---
2 - name: Create
3   hosts: localhost
4   gather_facts: false
5   tasks:

```

```

6      - name: Create podman container
7        containers.podman.podman_container:
8          name: vector-podman
9          image: docker.io/python:3.11-slim
10         state: started
11         command: sleep infinity
12         register: podman_container_result
13
14      - name: Assert container is running
15        ansible.builtin.assert:
16          that:
17            - podman_container_result.container.State.Running |
default(false)

```

- Molecule вызывает `create` ;
- playbook на `localhost` создает контейнер;
- проверяем, что он действительно запущен.
- создал файл `destroy.yml`

```

1  ---
2  - name: Destroy
3    hosts: localhost
4    gather_facts: false
5    tasks:
6      - name: Remove podman container
7        containers.podman.podman_container:
8          name: vector-podman
9          state: absent

```

- проверил на ошибки `podman-scenario` , исправил пути поиска роли. Эта же команда запуска облегченного сценария будет и в файле `tox.ini`

```

1  molecule test -s podman

```

Тест прошел успешно

```
vboxuser@KVA:~/ansible/vector-role$ molecule test -s podman
PLAY [Verify] *****
TASK [Gathering Facts] *****
ok: [vector-podman]

TASK [Check vector install directory] *****
ok: [vector-podman]

TASK [Check vector binary] *****
ok: [vector-podman]

TASK [Check vector config] *****
ok: [vector-podman]

TASK [Validate vector config] *****
ok: [vector-podman]

TASK [Assert vector role results] *****
ok: [vector-podman] => {
  "changed": false,
  "msg": "All assertions passed"
}

PLAY RECAP *****
vector-podman      : ok=6    changed=0    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0

INFO     podman -> verify: Executed: Successful
INFO     podman -> destroy: Executing

PLAY [Destroy] *****
TASK [Remove podman container] *****
changed: [localhost]

PLAY RECAP *****
localhost          : ok=1    changed=1    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0

INFO     podman -> destroy: Executed: Successful
INFO     podman -> scenario: Pruning extra files from scenario ephemeral directory
INFO     Molecule executed 1 scenario (1 successful)
```

- скачал **Tox** и запустил его

[illegible]

Для чистоты выполнения задания скачал указанный в задании образ `docker pull`

aragast/netology:latest

Запустил командой `docker run --privileged=True -v`

```
/home/vboxuser/ansible/vector-role:/opt/vector-role -w /opt/vector-role -  
it aragast/netology:latest /bin/bash
```

Подключился и сделал проверки версий ПО

```
vboxuser@VMA:~/ansible/vector-role$ docker run --privileged=True -v /home/vboxuser/ansible/vector-role:/opt/vector-role -w /opt/vector-role -it aragast/netology:latest /bin/bash  
[root@b74943a17d3d vector-role]# python3 --version  
Python 3.6.8  
[root@b74943a17d3d vector-role]# pip --version  
pip 21.3.1 from /usr/local/lib/python3.6/site-packages/pip (python 3.6)  
[root@b74943a17d3d vector-role]# tox --version  
3.25.0 imported from /usr/local/lib/python3.6/site-packages/tox/_init_.py  
[root@b74943a17d3d vector-role]# ansible --version  
bash: ansible: command not found  
[root@b74943a17d3d vector-role]# molecule --version  
bash: molecule: command not found  
[root@b74943a17d3d vector-role]# podman --version  
podman version 4.0.2  
[root@b74943a17d3d vector-role]# docker --version  
bash: docker: command not found  
[root@b74943a17d3d vector-role]# cat /etc/os-release  
NAME="Red Hat Enterprise Linux"  
VERSION="8.6 (ottpa)"  
ID="rhel"  
ID_LIKE="fedora"  
VERSION_ID="8.6"  
PLATFORM_ID="platform:el8"  
PRETTY_NAME="Red Hat Enterprise Linux 8.6 (ottpa)"  
ANSI_COLOR="0;31"  
CPE_NAME="cpe:/o:redhat:enterprise_linux:8::baseos"  
HOME_URL="https://www.redhat.com/"  
DOCUMENTATION_URL="https://access.redhat.com/documentation/red_hat_enterprise_linux/8/"  
BUG_REPORT_URL="https://bugzilla.redhat.com/"  
  
REDHAT_BUGZILLA_PRODUCT="Red Hat Enterprise Linux 8"  
REDHAT_BUGZILLA_PRODUCT_VERSION=8.6  
REDHAT_SUPPORT_PRODUCT="Red Hat Enterprise Linux"  
REDHAT_SUPPORT_PRODUCT_VERSION=8.6
```

Установил зависимости из `tox-requirements`

```
pip install "ansible<5.0" "molecule<4.0" "molecule-podman" jmespath lxml  
selinux
```

```
Successfully installed Jinja2-3.0.3 MarkupSafe-2.0.1 PyYAML-6.0.1 ansible-4.10.0 ansible-compat-1.0.0 ansible-core-2.11.12 arrow-1.2.3 bcrypt-4.0.1 binaryornot-0.4.4 cached-property-1.5.2 cerberu  
s-1.3.5 certifi-2025.4.26 cffi-1.15.1 charset-normalizer-2.0.12 click-8.0.4 click-help-colors-0.9.4 commonmark-0.9.2 cookiecutter-1.7.3 cryptography-40.0.2 dataclasses-0.8 enrich-1.2.7 Jinja2-tim  
e-0.2.0 jmespath-0.10.0 lxml-5.4.0 molecule-3.6.1 molecule-podman-1.1.0 paramiko-2.12.0 poyo-0.5.0 pycparser-2.21 pygments-2.14.0 pynacl-1.5.0 python-dateutil-2.9.0.post0 python-slugify-6.1.2 req  
uests-2.27.1 resolvelib-0.5.4 rich-12.6.0 subprocess-tee-0.3.5 text-unidecode-1.3  
WARNING: Running pip as the 'root' user can result in broken permissions and conflicting behaviour with the system package manager. It is recommended to use a virtual environment instead: https://  
/pip.pypa.io/warnings/venv  
[root@b74943a17d3d vector-role]#
```

Чтобы не ломать рабочий стек, который успешно отработал без скачанного образа, создал второй комплект файлов `tox-legacy.ini` и `tox-legacy-requirements.txt`

```
1  [tox]  
2  minversion = 3.20  
3  envlist = py36  
4  skipsdist = true  
5  
6  [testenv]  
7  basepython = python3.6  
8  passenv = *  
9  deps =  
10     -r tox-legacy-requirements.txt  
11  commands =  
12     {posargs:molecule test -s podman --destroy always}  
13  
14  ---  
15  ansible<5.0  
16  molecule<4.0  
17  molecule-podman  
18  jmespath  
19  lxml  
20  selinux  
21  
22  ---
```

Скорректировал файл `Converge.yml`

```

1  ---
2  - name: Converge
3    hosts: all
4    gather_facts: true
5    roles:
6      - role: "{{ lookup('env', 'MOLECULE_PROJECT_DIRECTORY') }}"

```

Запустил проверку командой

```
1  tox -c tox-legacy.ini
```

Тестирование прошло успешно

```

[root@74943a17d3d vector-role]# tox -c tox-legacy.ini
py36 installed: ansible==10.0,ansible-compat==1.0.0,ansible-core==2.11.12,arrow==1.2.3,bcrypt==4.0.1,binaryornot==0.4.4,cached-property==1.5.2,Cerberus==1.3.5,certifi==2025.4.26,cffi==1.15.1,charset-normalizer==2.0.12,click==8.0.4,click-help-colors==0.9.4,commonmark==0.9.2,cookiecutter==1.7.3,cryptography==40.0.2,dataclasses==0.8,distro==1.9.0,enrich==1.2.7,idna==3.10,importlib-metadata==4.8.3,jinja2==3.0.3,jinja2-time==0.2.0,jmespath==0.10.0,lxml==5.4.0,MarkupSafe==2.0.1,molecule==3.6.1,molecule-podman==1.1.0,packaging==21.3,paramiko==2.12.0,pluggy==1.0.0,poyo==0.5.0,pycparser==2.21,Pygments==2.14.0,PyNaCl==1.5.0,pyrsing==3.1.4,python-dateutil==2.9.0.post0,python-slugify==6.1.2,PyYAML==6.0.1,requests==2.27.1,resolvelib==0.5.4,rich==12.6.0,selinux==0.2.1,six==1.17.0,subprocess-tee==0.3.5,text-unidecode==1.3,typing_extensions==4.1.1,urllib3==1.26.20,zip==3.6.0
py36 run-test-pre: PYTHONHASHSEED='333596886'
py36 run-test: commands[0] | molecule test -s podman --destroy always
[DEPRECATION WARNING]: Ansible will require Python 3.8 or newer on the controller starting with Ansible 2.12. Current version: 3.6.8 (default, Jan 14 2022, 11:04:20) [GCC 8.5.0 20210514 (Red Hat 8.5.0-7)]. This feature will be removed from ansible-core in version 2.12. Deprecation warnings can be disabled by setting deprecation_warnings=False in ansible.cfg.
/opt/vector-role/.tox/py36/lib/python3.6/site-packages/ansible/parsing/vault/_init_.py:44: CryptographyDeprecationWarning: Python 3.6 is no longer supported by the Python core team. Therefore, support for it is deprecated in cryptography. The next release of cryptography will remove support for Python 3.6.
  from cryptography.exceptions import InvalidSignature
/opt/vector-role/.tox/py36/lib/python3.6/site-packages/requests/_init_.py:104: RequestsDependencyWarning: urllib3 (1.26.20) or chardet (5.0.0)/charset_normalizer (2.0.12) doesn't match a supported version!
  RequestsDependencyWarning)
INFO     podman scenario test matrix: create, converge, idempotence, verify, destroy
INFO     Performing prunrun...
INFO     Set ANSIBLE_LIBRARY=/root/.cache/ansible-compat/b984a4/modules:/root/.ansible/plugins/modules:/usr/share/ansible/plugins/modules
INFO     Set ANSIBLE_COLLECTIONS_PATH=/root/.cache/ansible-compat/b984a4/collections:/root/.ansible/collections:/usr/share/ansible/collections
INFO     Set ANSIBLE_ROLES_PATH=/root/.cache/ansible-compat/b984a4/roles:/root/.ansible/roles:/usr/share/ansible/roles:/etc/ansible/roles
INFO     Using /root/.ansible/roles/arturprozkov.vector_role symlink to current repository in order to enable Ansible to find the role using its expected full name.
INFO     Running podman > create
INFO     Sanity checks: 'podman'
[DEPRECATION WARNING]: Ansible will require Python 3.8 or newer on the controller starting with Ansible 2.12. Current version: 3.6.8 (default, Jan 14 2022, 11:04:20) [GCC 8.5.0 20210514 (Red Hat 8.5.0-7)]. This feature will be removed from ansible-core in version 2.12. Deprecation warnings can be disabled by setting deprecation_warnings=False in ansible.cfg.
/opt/vector-role/.tox/py36/lib/python3.6/site-packages/ansible/parsing/vault/_init_.py:44: CryptographyDeprecationWarning: Python 3.6 is no longer supported by the Python core team. Therefore, support for it is deprecated in cryptography. The next release of cryptography will remove support for Python 3.6.
  from cryptography.exceptions import InvalidSignature
[DEPRECATION WARNING]: Ansible will require Python 3.8 or newer on the controller starting with Ansible 2.12. Current version: 3.6.8 (default, Jan 14 2022, 11:04:20) [GCC 8.5.0 20210514 (Red Hat 8.5.0-7)]. This feature will be removed from ansible-core in version 2.12. Deprecation warnings can be disabled by setting deprecation_warnings=False in ansible.cfg.
PLAY [Create] *****
TASK [Create podman container] *****
/opt/vector-role/.tox/py36/lib/python3.6/site-packages/ansible/parsing/vault/_init_.py:44: CryptographyDeprecationWarning: Python 3.6 is no longer supported by the Python core team. Therefore, support for it is deprecated in cryptography. The next release of cryptography will remove support for Python 3.6.
  from cryptography.exceptions import InvalidSignature
changed: [localhost]

TASK [Assert container is running] *****
ok: [localhost] => {
  "changed": false,
  "msg": "All assertions passed"
}

PLAY RECAP *****
localhost                : ok=2    changed=1    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0

INFO     Running podman > converge
[DEPRECATION WARNING]: Ansible will require Python 3.8 or newer on the controller starting with Ansible 2.12. Current version: 3.6.8 (default, Jan 14 2022, 11:04:20) [GCC 8.5.0 20210514 (Red Hat 8.5.0-7)]. This feature will be removed from ansible-core in version 2.12. Deprecation warnings can be disabled by setting deprecation_warnings=False in ansible.cfg.
PLAY [Converge] *****
TASK [Gathering Facts] *****
/opt/vector-role/.tox/py36/lib/python3.6/site-packages/ansible/parsing/vault/_init_.py:44: CryptographyDeprecationWarning: Python 3.6 is no longer supported by the Python core team. Therefore, support for it is deprecated in cryptography. The next release of cryptography will remove support for Python 3.6.
  from cryptography.exceptions import InvalidSignature
ok: [vector-podman]

TASK [/opt/vector-role : Create vector install directory] *****
changed: [vector-podman]

TASK [/opt/vector-role : Get Vector distrib] *****
changed: [vector-podman]

TASK [/opt/vector-role : Unarchive Vector] *****
changed: [vector-podman]

TASK [/opt/vector-role : Create vector data directory] *****
changed: [vector-podman]

TASK [/opt/vector-role : Create vector config directory] *****
ok: [vector-podman]

TASK [/opt/vector-role : Deploy vector config] *****

```

```

changed: [vector-podman]

RUNNING HANDLER [/opt/vector-role : Validate vector config] *****
ok: [vector-podman]

PLAY RECAP *****
vector-podman      : ok=8  changed=5  unreachable=0  failed=0  skipped=0  rescued=0  ignored=0

INFO  Running podman > idempotence
[DEPRECATION WARNING]: Ansible will require Python 3.8 or newer on the controller starting with Ansible 2.12. Current version: 3.6.8 (default, Jan 14 2022, 11:04:20) [GCC 8.5.0 20210514 (Red Hat 8.5.0-7)]. This feature will be removed from ansible-core in version 2.12. Deprecation warnings can be disabled by setting deprecation_warnings=False in ansible.cfg.

PLAY [Converge] *****

TASK [Gathering Facts] *****
/opt/vector-role/.tox/py36/lib/python3.6/site-packages/ansible/parsing/vault/_init_.py:44: CryptographyDeprecationWarning: Python 3.6 is no longer supported by the Python core team. Therefore, support for it is deprecated in cryptography. The next release of cryptography will remove support for Python 3.6.
  from cryptography.exceptions import InvalidSignature
ok: [vector-podman]

TASK [/opt/vector-role : Create vector install directory] *****
ok: [vector-podman]

TASK [/opt/vector-role : Get Vector distrib] *****
ok: [vector-podman]

TASK [/opt/vector-role : Unarchive Vector] *****
skipping: [vector-podman]

TASK [/opt/vector-role : Create vector data directory] *****
ok: [vector-podman]

TASK [/opt/vector-role : Create vector config directory] *****
ok: [vector-podman]

TASK [/opt/vector-role : Deploy vector config] *****
ok: [vector-podman]

PLAY RECAP *****
vector-podman      : ok=6  changed=0  unreachable=0  failed=0  skipped=1  rescued=0  ignored=0

INFO  Idempotence completed successfully.
INFO  Running podman > verify
INFO  Running Ansible Verifier

[DEPRECATION WARNING]: Ansible will require Python 3.8 or newer on the controller starting with Ansible 2.12. Current version: 3.6.8 (default, Jan 14 2022, 11:04:20) [GCC 8.5.0 20210514 (Red Hat 8.5.0-7)]. This feature will be removed from ansible-core in version 2.12. Deprecation warnings can be disabled by setting deprecation_warnings=False in ansible.cfg.

PLAY [Verify] *****

TASK [Gathering Facts] *****
/opt/vector-role/.tox/py36/lib/python3.6/site-packages/ansible/parsing/vault/_init_.py:44: CryptographyDeprecationWarning: Python 3.6 is no longer supported by the Python core team. Therefore, support for it is deprecated in cryptography. The next release of cryptography will remove support for Python 3.6.
  from cryptography.exceptions import InvalidSignature
ok: [vector-podman]

TASK [Check vector install directory] *****
ok: [vector-podman]

TASK [Check vector binary] *****
ok: [vector-podman]

TASK [Check vector config] *****
ok: [vector-podman]

TASK [Validate vector config] *****
ok: [vector-podman]

TASK [Assert vector role results] *****
ok: [vector-podman] => {
  "changed": false,
  "msg": "All assertions passed"
}

PLAY RECAP *****
vector-podman      : ok=6  changed=0  unreachable=0  failed=0  skipped=0  rescued=0  ignored=0

INFO  Verifier completed successfully.
INFO  Running podman > destroy
[DEPRECATION WARNING]: Ansible will require Python 3.8 or newer on the controller starting with Ansible 2.12. Current version: 3.6.8 (default, Jan 14 2022, 11:04:20) [GCC 8.5.0 20210514 (Red Hat 8.5.0-7)]. This feature will be removed from ansible-core in version 2.12. Deprecation warnings can be disabled by setting deprecation_warnings=False in ansible.cfg.

PLAY [Destroy] *****

TASK [Remove podman container] *****
/opt/vector-role/.tox/py36/lib/python3.6/site-packages/ansible/parsing/vault/_init_.py:44: CryptographyDeprecationWarning: Python 3.6 is no longer supported by the Python core team. Therefore, support for it is deprecated in cryptography. The next release of cryptography will remove support for Python 3.6.
  from cryptography.exceptions import InvalidSignature
changed: [localhost]

PLAY RECAP *****
localhost          : ok=1  changed=1  unreachable=0  failed=0  skipped=0  rescued=0  ignored=0

INFO  Pruning extra files from scenario ephemeral directory

summary
-----
py36: commands succeeded
congratulations :)
[root@b74943a17d3d vector-role]#

```

Запустил все получившееся в гитхаб репозиторий с тегом 0.0.3

```

1  git status
2  git add molecule/podman tox-legacy.ini tox-legacy-requirements.txt
   tox.ini tox-requirements.txt
3  git commit -m "Add podman molecule scenario and legacy tox config"
4  git push origin main
5  git tag -a 0.0.3 -m "0.0.3"
6  git push origin 0.0.3

```


Необязательную часть не делал